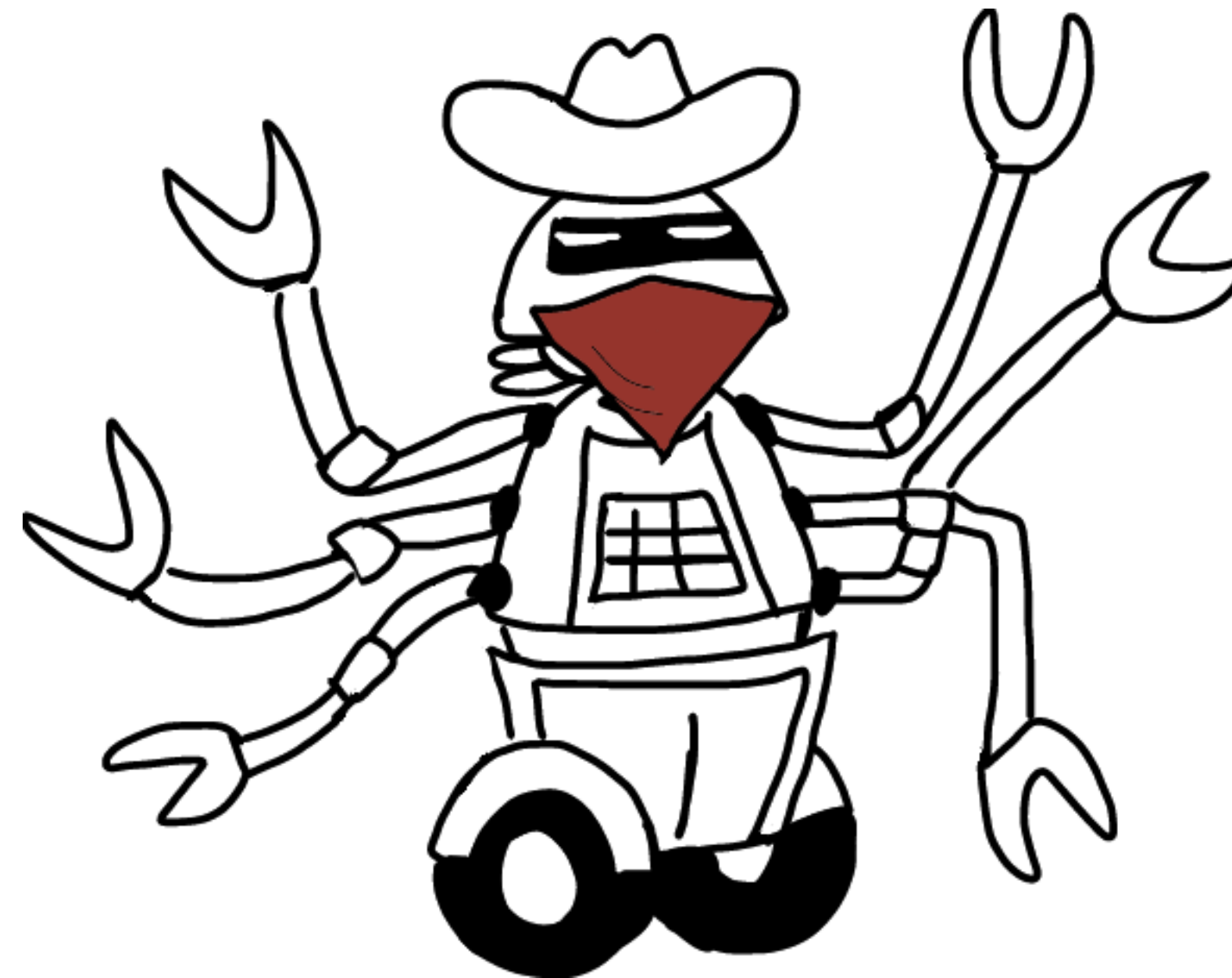


Reinforcement Learning

HSE, winter - spring 2025

Lecture 8: Multi-armed Bandits



Sergei Laktionov
slaktionov@hse.ru
[LinkedIn](#)

Let's Take A Step Back...

Initially, we treated RL problem differently because of three main pillars:

- Reward Function (Feedback) instead of directly provided correct labels
- Exploration Vs Exploitation
- Our actions affect the data we collect

Theorem (Bellman):

Policy is optimal $\iff$ it is greedy w.r.t. its induced Value function

$$\pi^*(s) \in \operatorname{argmax}_a [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s')] = \operatorname{argmax}_a Q^*(s, a)$$

Let's Take A Step Back...

Initially, we treated RL problem differently because of three main pillars:

- Reward Function (Feedback) instead of directly provided correct labels
- Exploration Vs Exploitation
- ~~Our actions affect the data we collect~~ $\Rightarrow$

Bandits

Theorem (Bellman):

Policy is optimal $\iff$ it is greedy w.r.t. its induced Value function

$$\pi^*(s) \in \operatorname{argmax}_a [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s')] = \operatorname{argmax}_a r(s, a)$$

Bandits: Problem Formulation

- We start from some initial state $s_t \sim \rho(\cdot | X)$ (can be fixed $s \sim \delta_{s_0}$)
- We get some observation $x_t \sim \mu(\cdot | s_t)$
- We do not know the true d.g.p. behind the reward function $r(s_t, a_t)$
- Our task is to choose an action to maximize G_0 from **repeated experiments**

$$\pi^*(s) \in \operatorname{argmax}_a [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s')] = \operatorname{argmax}_a r(s, a)$$
$$\hat{\pi}(s) \in \hat{r}(s, a)$$

Bandits: Problem Formulation

Q: So, is it just a classical supervised learning?

Bandits: Problem Formulation

Q: So, is it just a classical supervised learning?

Not quite:

- We still have **reward function**

Bandits: Problem Formulation

Q: So, is it just a classical supervised learning?

Not quite:

- We still have **reward function**
- Still need to collect the samples ourselves => **exploration-exploitation trade-off**

Two Ways to View Bandits

1. Repeated One-Stage RL:

- We have interactions with the environment that end after first round (all actions transition us into terminal state directly);
- Our goal is long-term average reward from these repeated experiments.

2. Action-Independent State Transitions:

- We have a classical RL setting, but our actions do not affect state transitions;
- One can view this as state transitions are generated by the environment only, while our actions affect only the immediate reward.

Idea: Mathematical treatment of both approaches is identical, but one can recognize Bandit tasks in a lot of different applications

Next Up: Start Simple

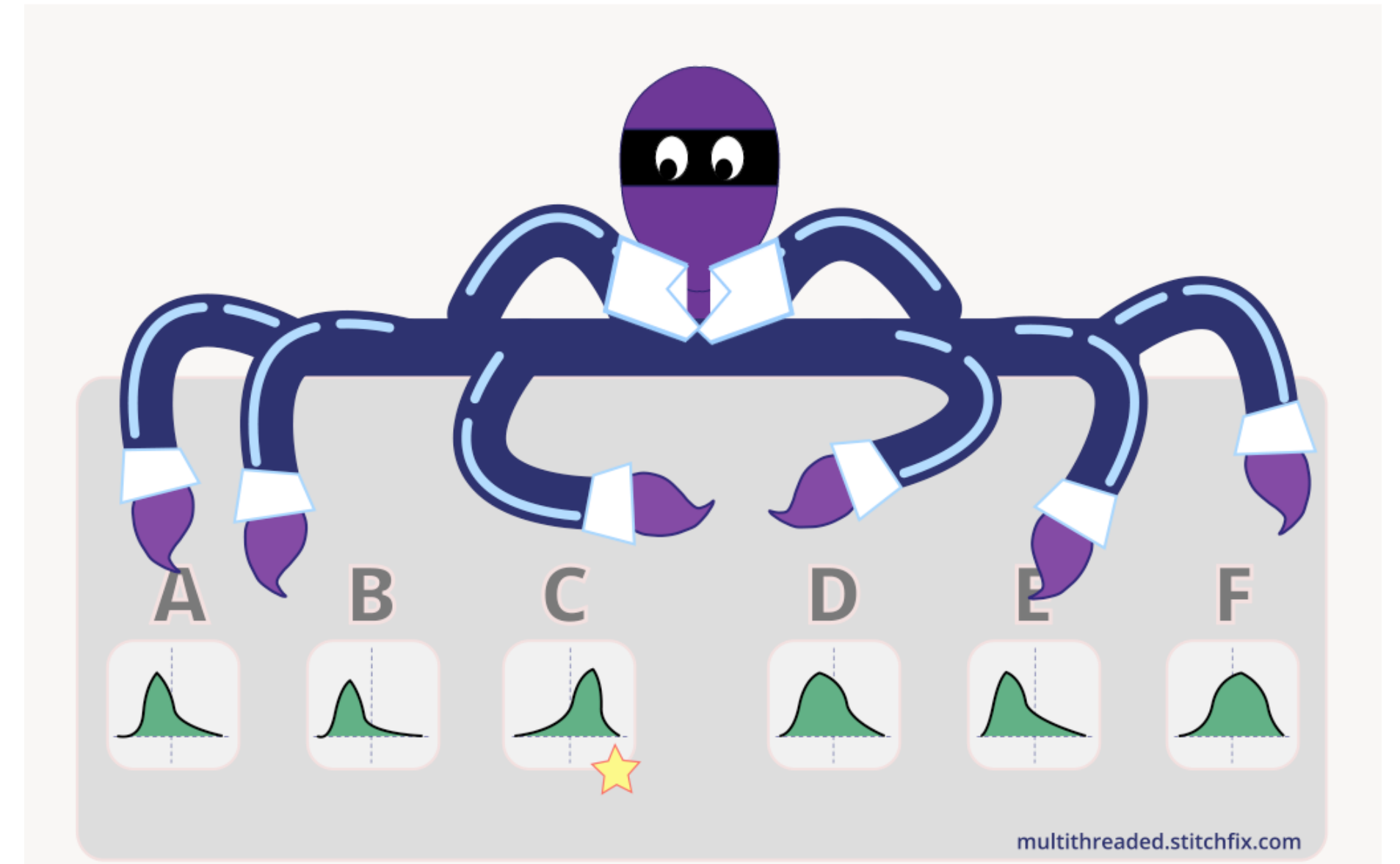
Multi-armed Bandit

- The initial state is fixed;
- The episode ends after the first step so we have a single state in the environment;
- An agent is facing repeatedly a choice among K different actions;
- We do not know the reward distribution for each of the K actions.

Multi-armed Bandit

- $\{p(r | a) | a \in \mathcal{A}\}$ is a set of reward's distributions;
- On each step, t an agent chooses a_t and get reward $r_t \sim p(\cdot | a_t)$

The agent's goal is to maximise $\mathbb{E}_{r \sim p(\cdot | a)}[\sum_{t=1}^T r_t]$ by choosing an action at each step.



Source

Examples

- News website: pick an article to show to a user to maximize the total number of clicks.
- Medical trials: pick treatment for a patient to cure as much people as possible.

For both examples $r \sim \text{Bernoulli}(p_a)$ with the unknown probability for each arm.

RL Formalism

- **Policy:** π is just a rule of making decisions on each step
- **Action value function:** $Q(a) = \mathbb{E}[r_t \mid a_t = a] = \mathbb{E}_{a \sim \pi(\cdot)}[\mathbb{E}_{r \sim p(\cdot|a)} r]$
- **Optimal value:** $V^* = \max_a Q(a)$
- **Gap:** $V^* - Q(a) \geq 0$
- **Total Regret:** $\mathbb{E} \sum_{t=1}^T [V^* - Q(a_t)] \rightarrow \min_{\pi}$

RL Formalism

- **Policy:** π is just a rule of making decisions on each step
- **Action value function:** $Q(a) = \mathbb{E}[r_t \mid a_t = a] = \mathbb{E}_{a \sim \pi(\cdot)}[\mathbb{E}_{r \sim p(\cdot|a)} r]$
- **Optimal value:** $V^* = \max_a Q(a)$
- **Gap:** $V^* - Q(a) \geq 0$
- **Total Regret:** $\mathbb{E} \sum_{t=1}^T [V^* - Q(a_t)] \rightarrow \min_{\pi} \iff \mathbb{E}_{a_t \sim \pi(\cdot), r_t \sim p(\cdot|a_t)} \sum_{t=1}^T r_t \rightarrow \max_{\pi}$

RL Formalism

- **Policy:** π is just a rule of making decisions on each step
- **Action value function:** $Q(a) = \mathbb{E}[r_t \mid a_t = a] = \mathbb{E}_{a \sim \pi(\cdot)}[\mathbb{E}_{r \sim p(\cdot|a)} r]$
- **Optimal value:** $V^* = \max_a Q(a)$
- **Gap:** $V^* - Q(a) \geq 0$
- **Total Regret:** $\mathbb{E} \sum_{t=1}^T [V^* - Q(a_t)] \rightarrow \min_{\pi} \iff \mathbb{E}_{a_t \sim \pi(\cdot), r_t \sim p(\cdot|a_t)} \sum_{t=1}^T r_t \rightarrow \max_{\pi}$

NB: V^* is a constant w.r.t. our policy and is determined solely by the state

RL Formalism

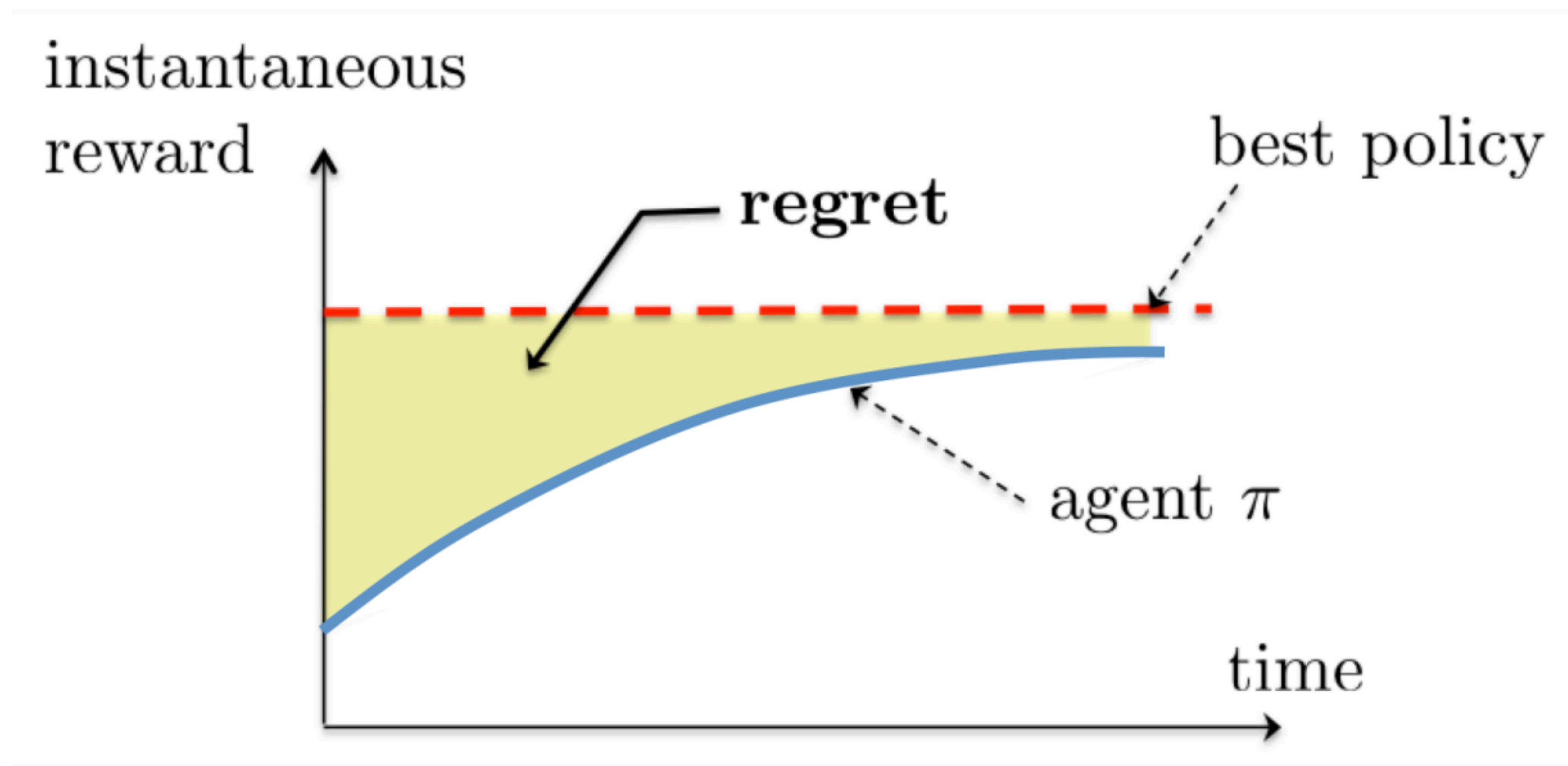
- **Policy:** π is just a rule of making decisions on each step
- **Action value function:** $Q(a) = \mathbb{E}[r_t | a_t = a] = \mathbb{E}_{a \sim \pi(\cdot)}[\mathbb{E}_{r \sim p(\cdot|a)} r]$
- **Optimal value:** $V^* = \max_a Q(a)$
- **Gap:** $V^* - Q(a) \geq 0$
- **Total Regret:** $\mathbb{E} \sum_{t=1}^T [V^* - Q(a_t)] \rightarrow \min_{\pi} \iff \mathbb{E}_{a_t \sim \pi(\cdot), r_t \sim p(\cdot|a_t)} \sum_{t=1}^T r_t \rightarrow \max_{\pi}$

NB: V^* is a constant w.r.t. our policy and is determined solely by the state

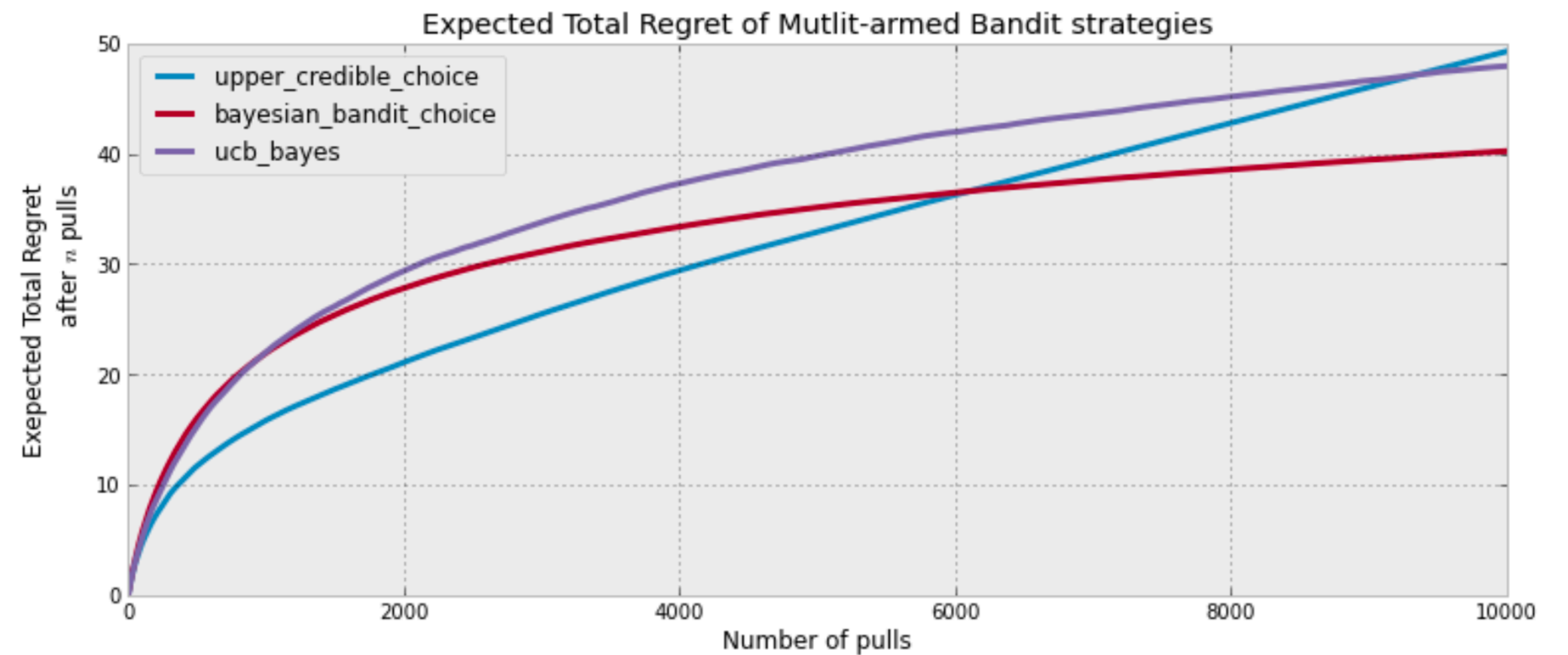
NB: V^* is a constant w.r.t. optimization, but it can be very important for normalization in the environments with different possible states (see later)

Regret Minimisation

$$\mathbb{E} \sum_{t=1}^T [V^* - Q(a_t)] \rightarrow \min_{\pi} \iff \mathbb{E}_{a_t \sim \pi(\cdot), r_t \sim p(\cdot|a_t)} \sum_{t=1}^T r_t \rightarrow \max_{\pi}$$



Source



Source

Regret

$$\sum_{t=1}^T [V^* - Q(a_t)] = TV^* - \sum_{t=1}^T Q(a_t)$$

Realised regret: $R(T) = TV^* - \sum_{t=1}^T Q(a_t)$

Expected regret: $\mathbb{E}[R(T)]$

We are interested in the minimising of $\lim_{T \rightarrow \infty} \mathbb{E}[R(T)]$

Regret Bounds

$$n_T(a) = \sum_{t=1}^T \mathbb{I}[a_t = a]$$

$$R(T) = \sum_a n_T(a) [V^* - Q(a)]$$

Upper bound: $\mathbb{E}[R(T)] \leq T(V^* - \min_a Q(a))$

Regret Bounds

$$n_T(a) = \sum_{t=1}^T \mathbb{I}[a_t = a]$$

$$R(T) = \sum_a n_T(a) [V^* - Q(a)]$$

Upper bound: $\mathbb{E}[R(T)] \leq T(V^* - \min_a Q(a))$

- Our goal is to get a *sublinear regret* $R_T/T \rightarrow 0$
- It means that our algorithm makes a decreasing number of mistakes as time evolves
- This implies $\mathbb{E}[V^* - V^\pi] \rightarrow 0$

Regret Bounds

$$n_T(a) = \sum_{t=1}^T \mathbb{I}[a_t = a]$$

$$R(T) = \sum_a n_T(a) [V^* - Q(a)]$$

Upper bound: $\mathbb{E}[R(T)] \leq T(V^* - \min_a Q(a))$

Lower bound (Lai-Robbins Theorem):

$$\mathbb{E}[R(T)] \geq \log T \sum_{a|V^* > Q(a)} \frac{V^* - Q(a)}{D_{KL}(p(r|a) || p(r|a^*))}$$

Action Values

$$Q_t(a) = \frac{\sum_{n=1}^t \mathbb{I}(a_n = a) r_n}{\sum_{n=1}^t \mathbb{I}(a_n = a)} = \frac{\sum_{n=1}^t \mathbb{I}(a_n = a) r_n}{n_t(a)} \iff$$

Action Values

$$Q_t(a) = \frac{\sum_{n=1}^t \mathbb{I}(a_n = a) r_n}{\sum_{n=1}^t \mathbb{I}(a_n = a)} = \frac{\sum_{n=1}^t \mathbb{I}(a_n = a) r_n}{n_t(a)} \iff \begin{aligned} Q_t(a) &= Q_{t-1}(a) + \alpha_t(a) [r_t - Q_{t-1}(a)] \\ \alpha_t(a) &= \frac{\mathbb{I}[a_t = a]}{n_t(a)}, n_t(a) = n_{t-1}(a) + \mathbb{I}[a_t = a] \end{aligned}$$

Explore-first

- Explore each arm for N times
- Select arm greedily w.r.t. current approximation of Q
- For $N = T^{\frac{1}{3}}(\log T)^{\frac{2}{3}}$: $\mathbb{E}[R(T)] \leq O(T^{\frac{2}{3}}(\log T)^{\frac{1}{3}})$

ε -greedy Policy

$$\pi_t(a) = \begin{cases} (1 - \varepsilon) + \frac{\varepsilon}{|\mathcal{A}|}, & \text{if } a = \operatorname{argmax}_a Q_t(a) \\ \frac{\varepsilon}{|\mathcal{A}|}, & \text{otherwise} \end{cases}$$

- Greedy policy can stuck in a suboptimal action forever
- ε -greedy continues to explore

ϵ -greedy Policy

$$\pi_t(a) = \begin{cases} (1 - \epsilon) + \frac{\epsilon}{|\mathcal{A}|}, & \text{if } a = \operatorname{argmax}_a Q_t(a) \\ \frac{\epsilon}{|\mathcal{A}|}, & \text{otherwise} \end{cases}$$

- Greedy policy can stuck in a suboptimal action forever
- ϵ -greedy continues to explore

ϵ -greedy policy has linear regret

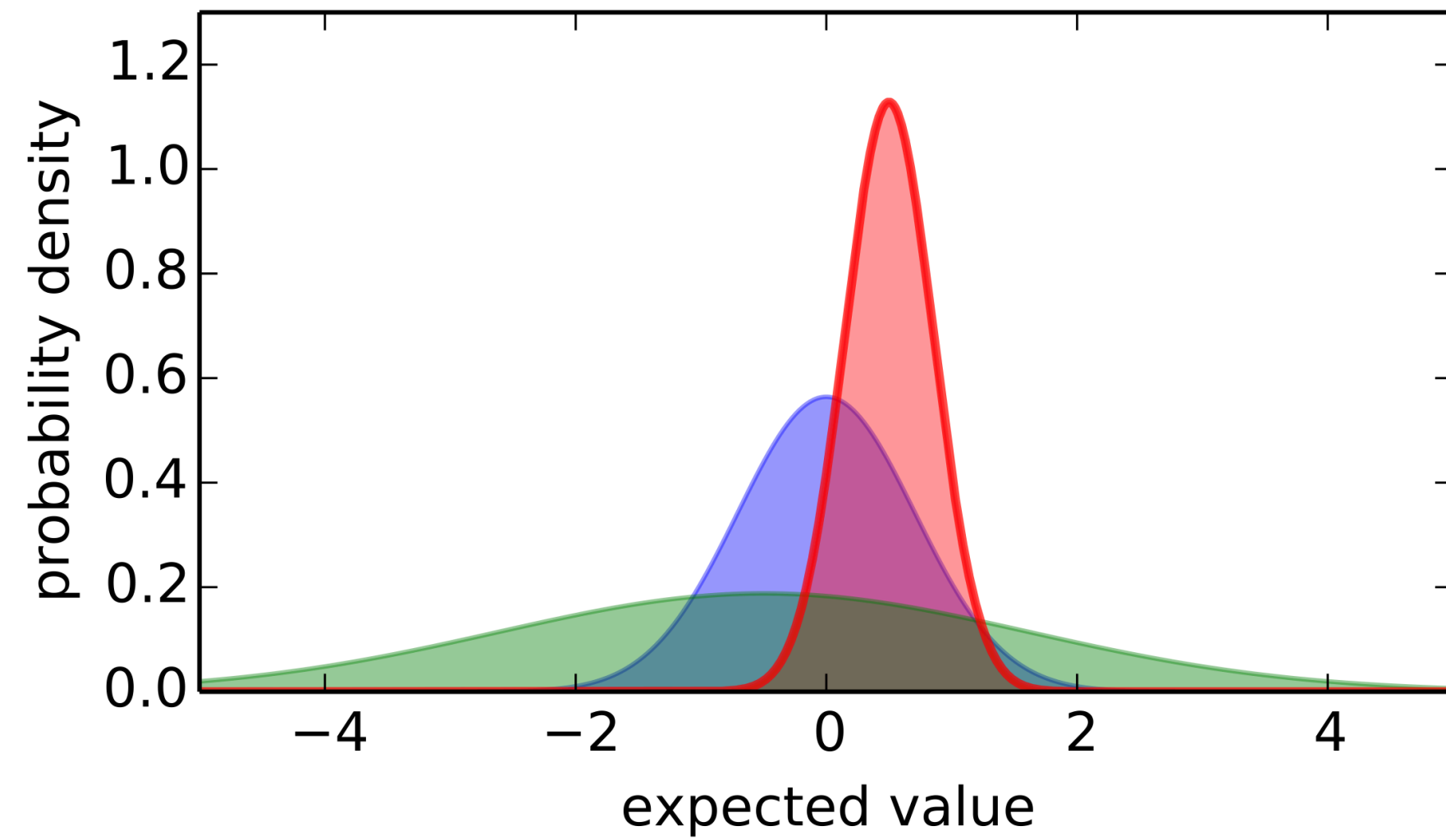
Adaptive Exploration

Epsilon-greedy algorithm with exploration probabilities $\varepsilon_t = t^{-\frac{1}{3}}(K \log T)^{\frac{1}{3}}$ achieves regret bound:

$$\mathbb{E}[R(T)] \leq T^{\frac{2}{3}} O(K \log T)^{\frac{1}{3}}$$

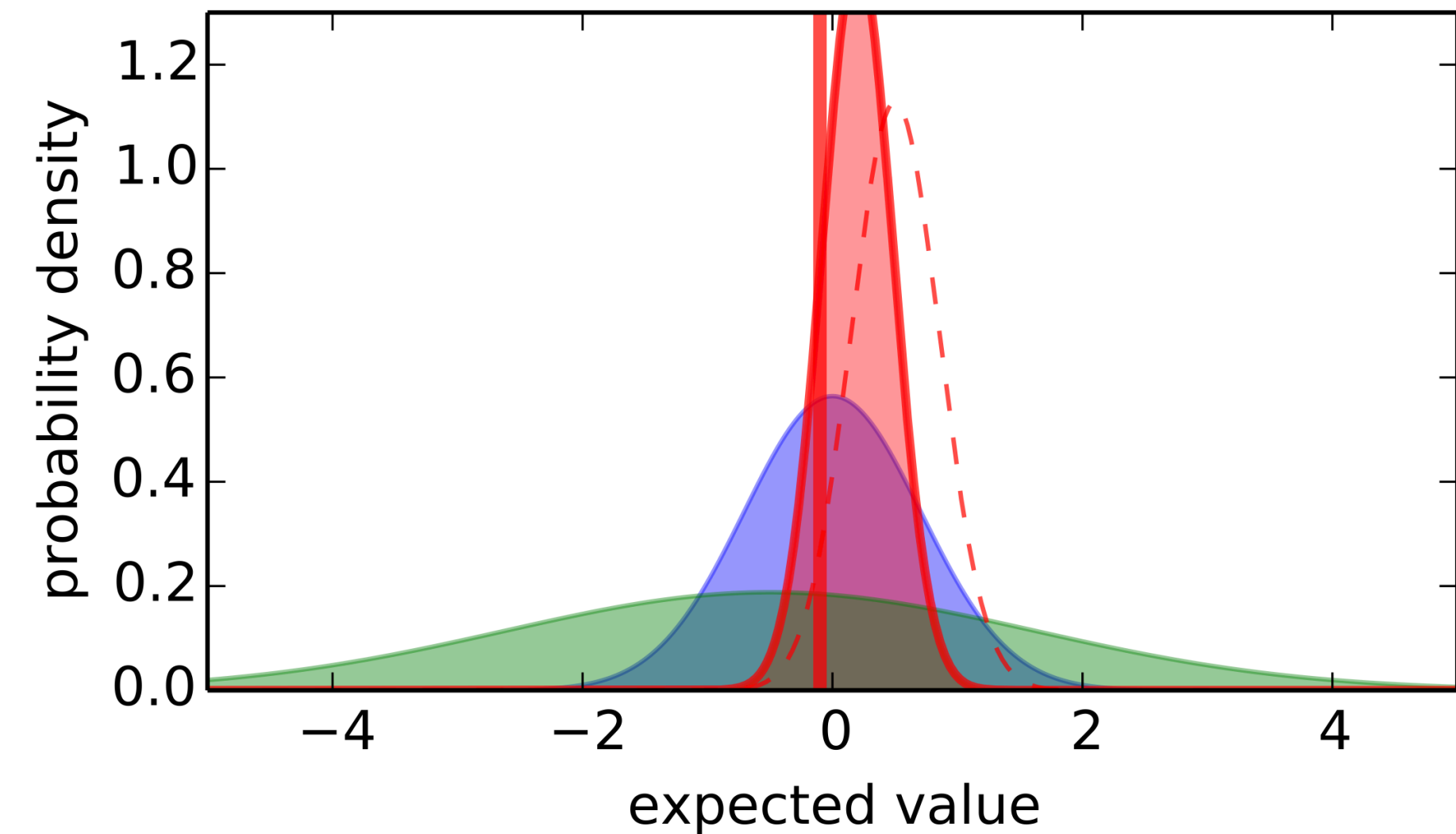
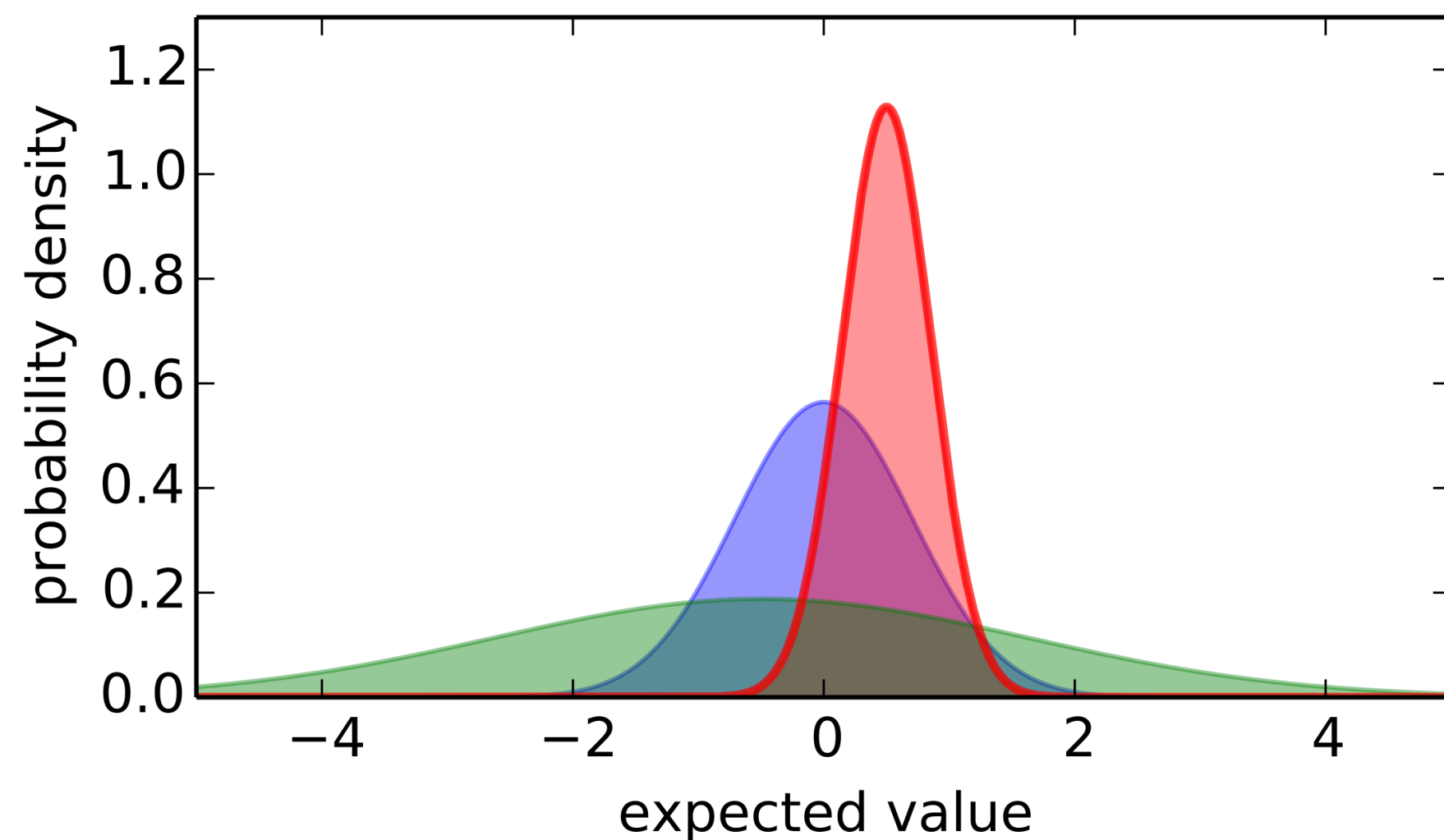
Next Up: Let's Improve

Optimism In The Face of Uncertainty



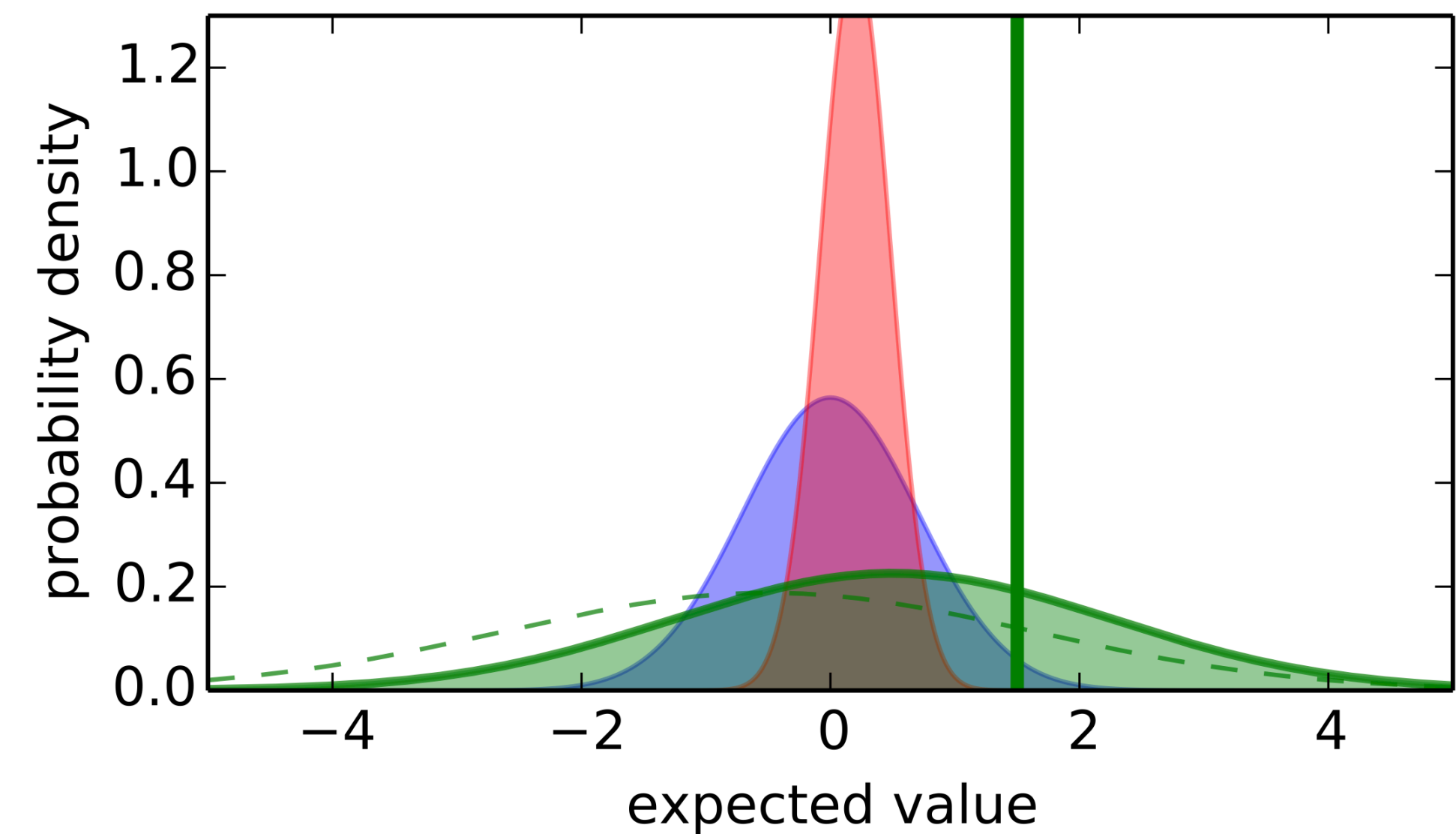
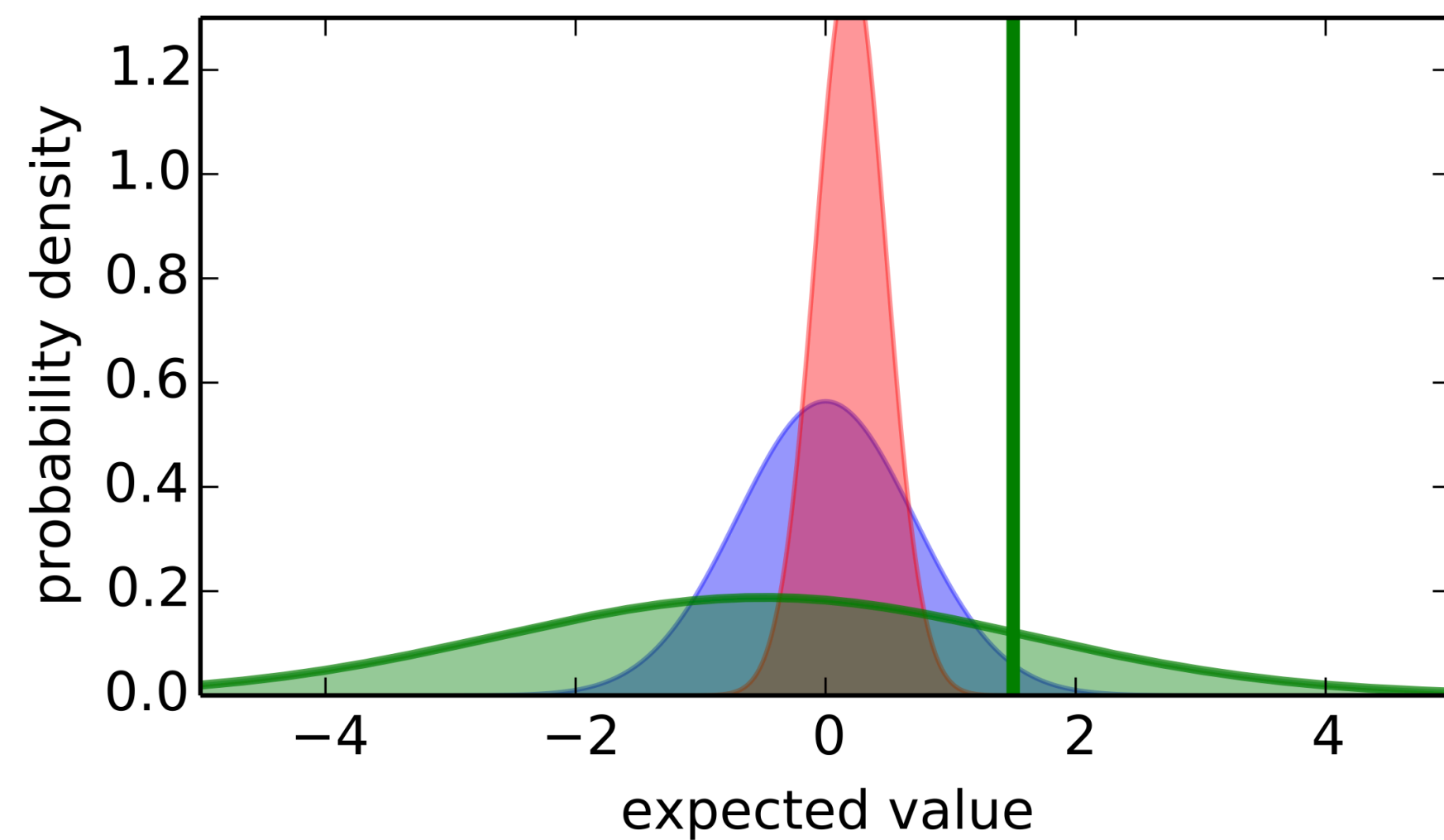
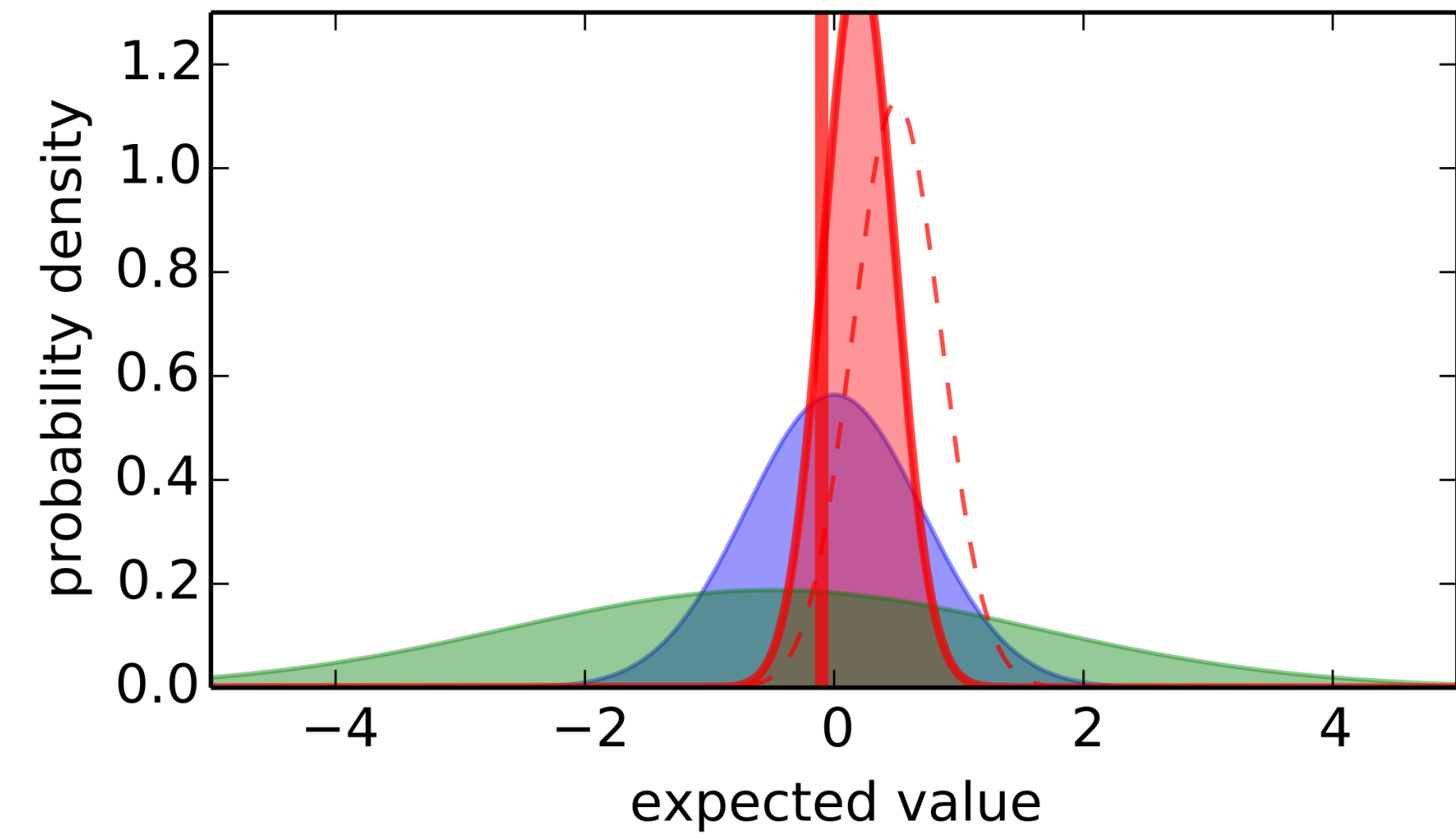
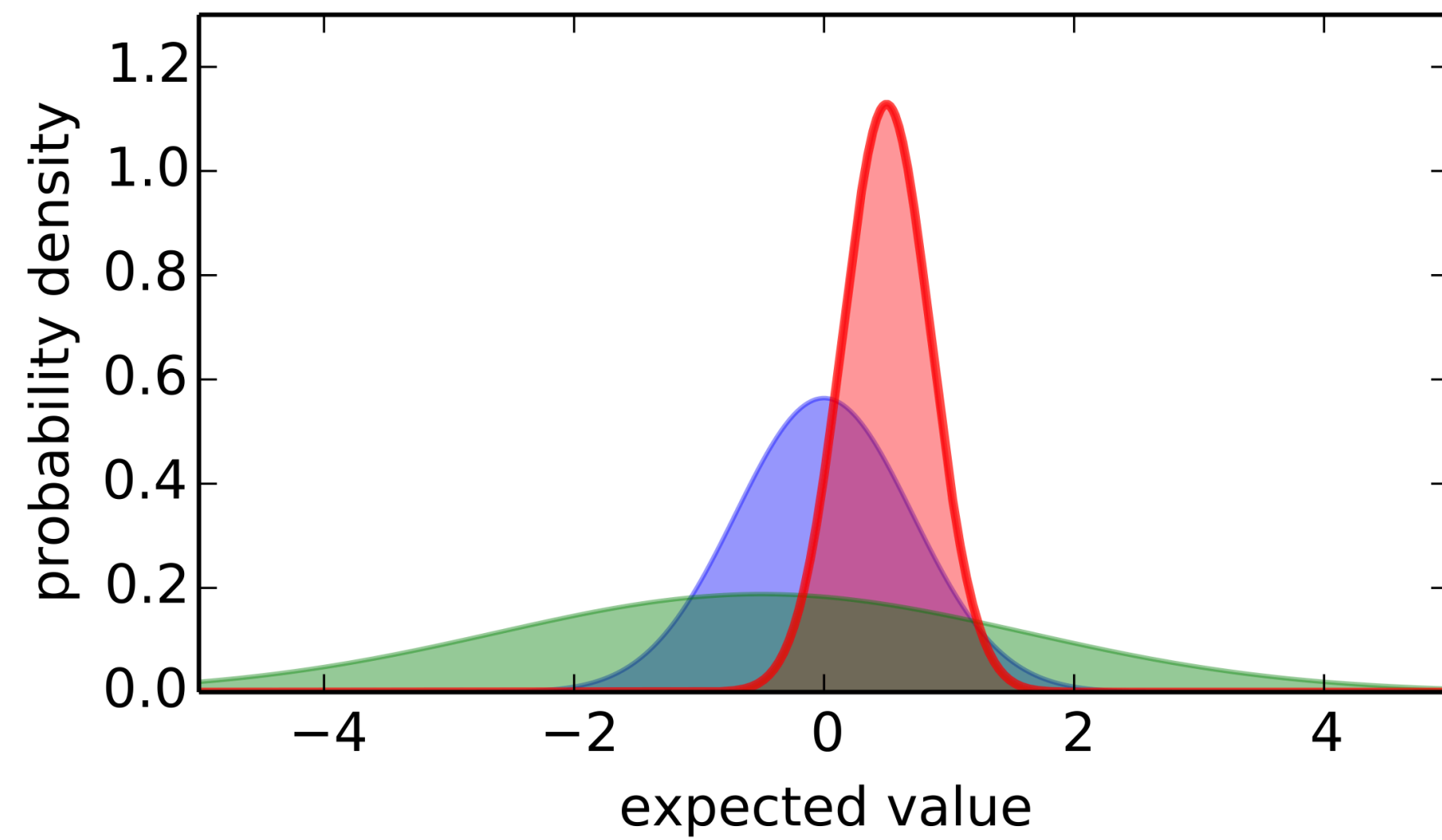
- Which action should we pick?

Optimism In The Face of Uncertainty



- Which action should we pick?
- The more uncertain we are about an action-value, the more critical it is to explore that action.
- It could be the best action.

Optimism In The Face of Uncertainty

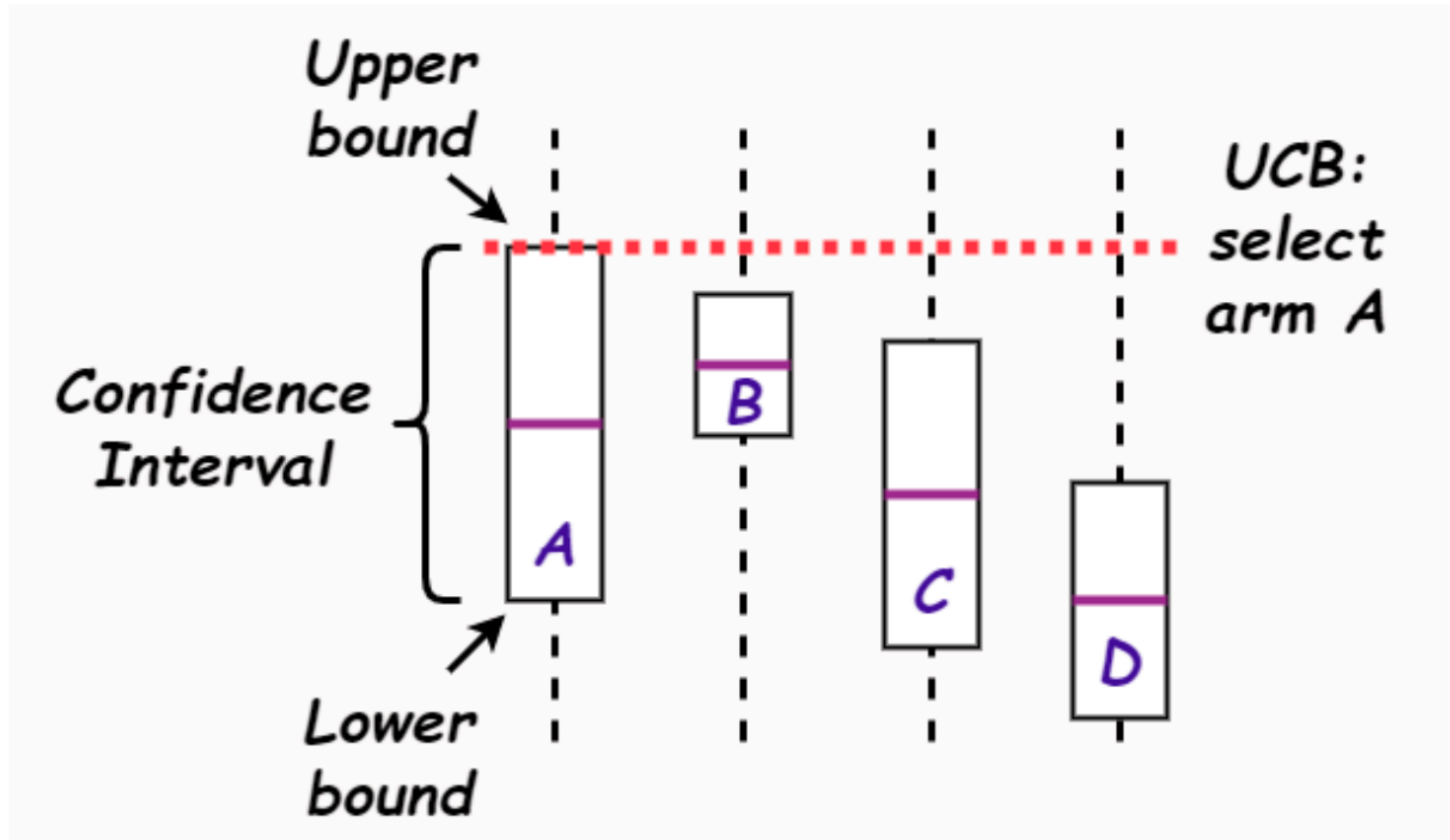


Source

Upper Confidence Bound

- Estimate an upper confidence $U_t(a)$ for each action value, such that $Q(a) \leq Q_t(a) + U_t(a)$ with high probability.
- This depends on the number of times $n_t(a)$ has been selected
 - Small $n_t(a) \Rightarrow$ large $U_t(a)$ (estimated value is uncertain)
 - Large $n_t(a) \Rightarrow$ small $U_t(a)$ (estimated value is accurate)
- Select action maximising upper confidence bound (UCB):
$$a_t = \operatorname{argmax}_{a \in A} [Q_t(a) + U_t(a)]$$

Illustration



Source: RL Textbook

Concentration Inequalities

$$\mathbb{P}(|\bar{X}_t - \mu| \leq \text{"small"}) \geq 1 - \text{"small"}.$$

Hoeffding's Inequality

$$\mathbb{P}(|\bar{X}_t - \mu| \leq \text{"small"}) \geq 1 - \text{"small"}.$$

Let $X_1, \dots, X_t$ be i.i.d. random variables in $[0,1]$ with true mean μ , and let $\bar{X}_t$ be the sample mean. Then $\mathbb{P}(|\bar{X}_t - \mu| \leq u) \geq 1 - e^{-2tu^2}$.

Hoeffding's Inequality

$$\mathbb{P}(|\bar{X}_t - \mu| \leq \text{"small"}) \geq 1 - \text{"small"}.$$

Let $X_1, \dots, X_t$ be i.i.d. random variables in $[0, 1]$ with true mean μ , and let $\bar{X}_t$ be the sample mean. Then $\mathbb{P}(|\bar{X}_t - \mu| \leq u) \geq 1 - e^{-2tu^2}$.

$$\mathbb{P}(Q_t(a) + U_t(a) \leq Q(a)) \leq e^{-2n_t(a)U_t(a)^2}$$

UCB

- Estimate an upper confidence $U_t(a)$ for each action value, such that $Q(a) \leq Q_t(a) + U_t(a)$ with high probability.

$$\mathbb{P}(Q_t(a) + U_t(a) \leq Q(a)) \leq e^{-2n_t(a)U_t(a)^2} = p$$

$$\text{If } U_t(a) = \sqrt{\frac{-\log p}{2n_t(a)}} \text{ then } e^{-2n_t(a)U_t(a)^2} = p$$

$$\text{Reduce } p \text{ as we get more information: } p = \frac{1}{t^{2C}}$$

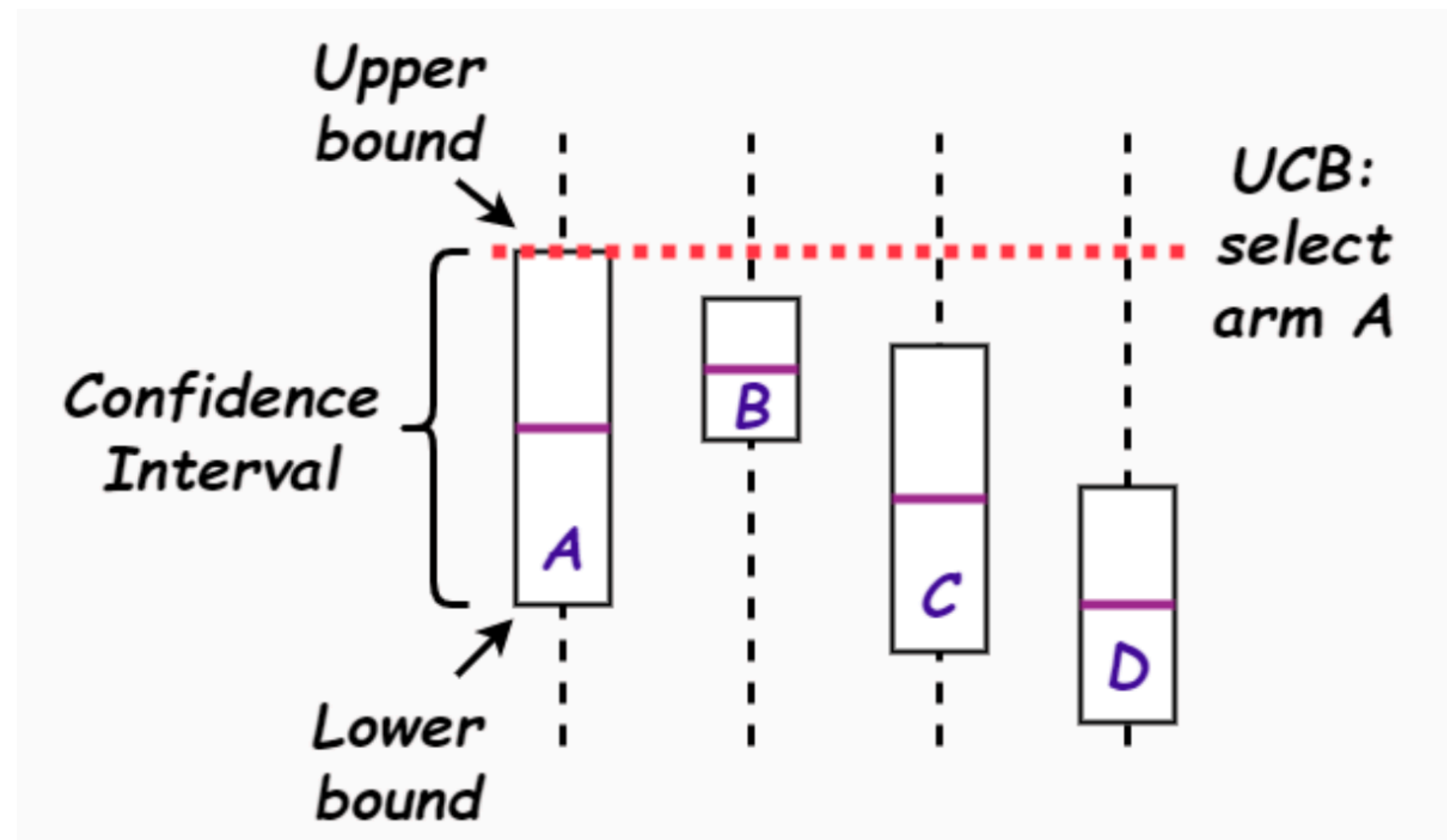
$$U_t(a) = C \sqrt{\frac{\log t}{n_t(a)}}$$

UCB

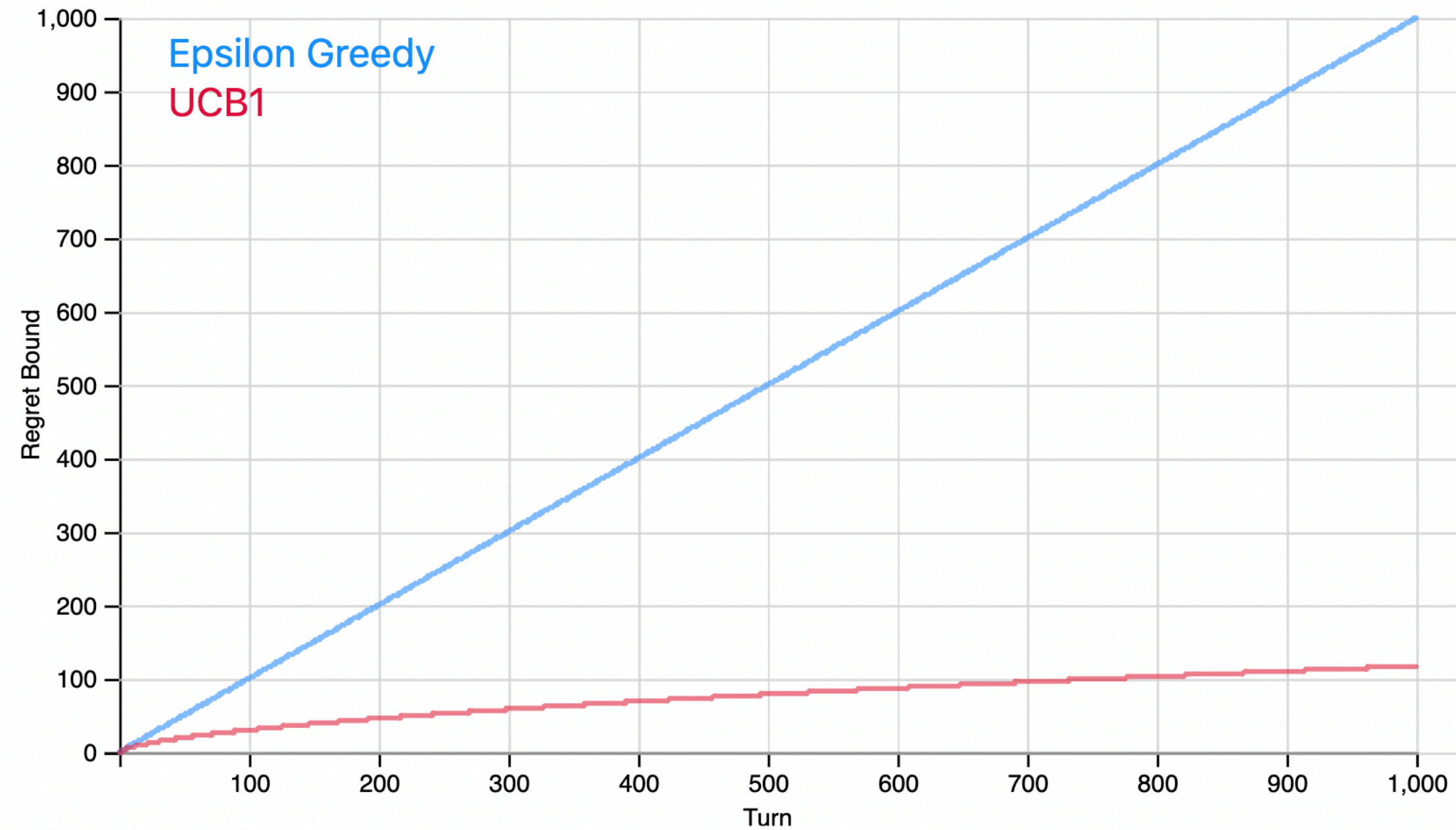
- Select action maximising upper confidence bound (UCB):

$$a_t = \operatorname{argmax}_{a \in \mathcal{A}} \left[Q_t(a) + C \sqrt{\frac{\log t}{n_t(a)}} \right]$$

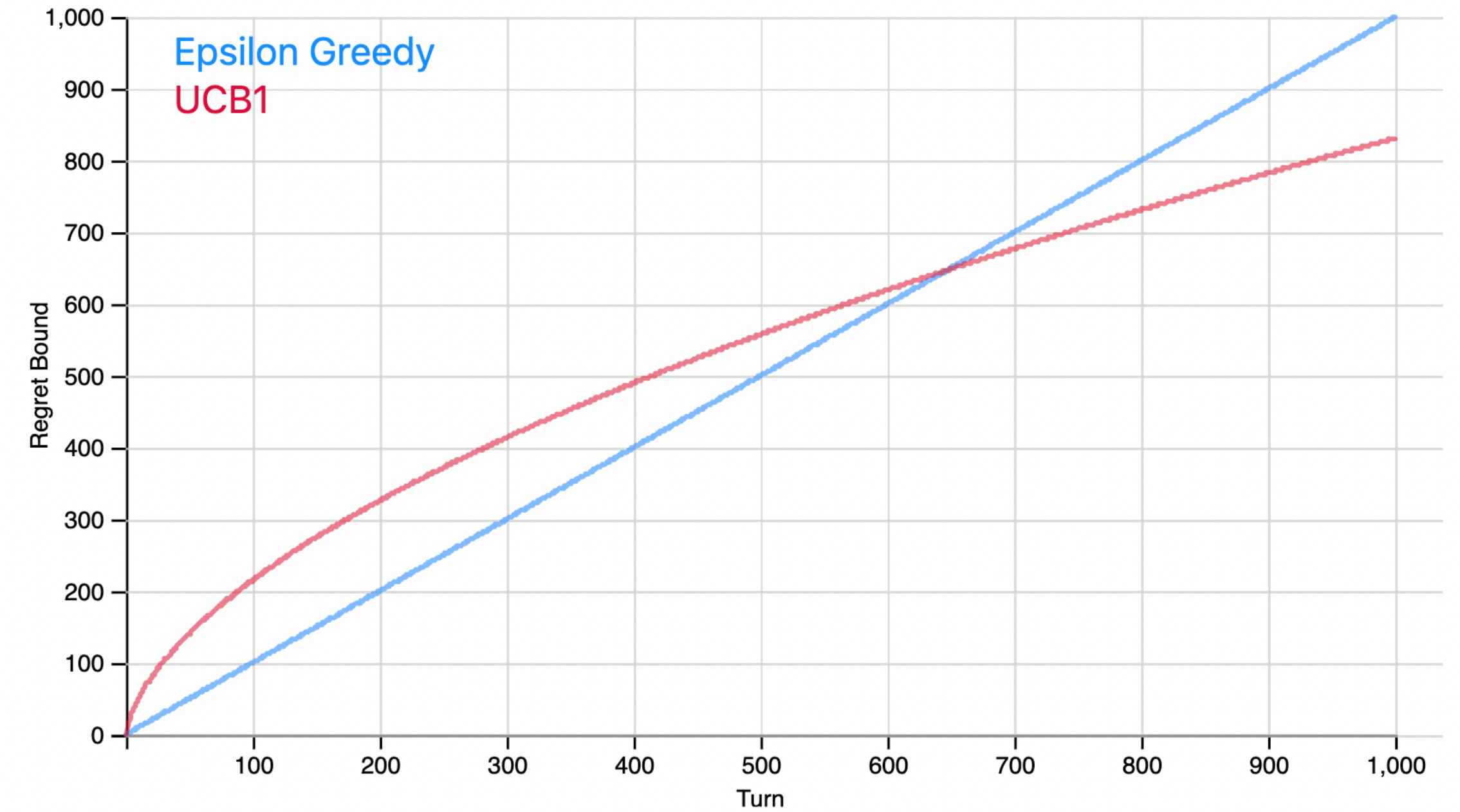
- Theorem: if $C = 1$ then UCB achieves logarithmic expected regret.



Comparison



k (number of arms): T (number of steps):

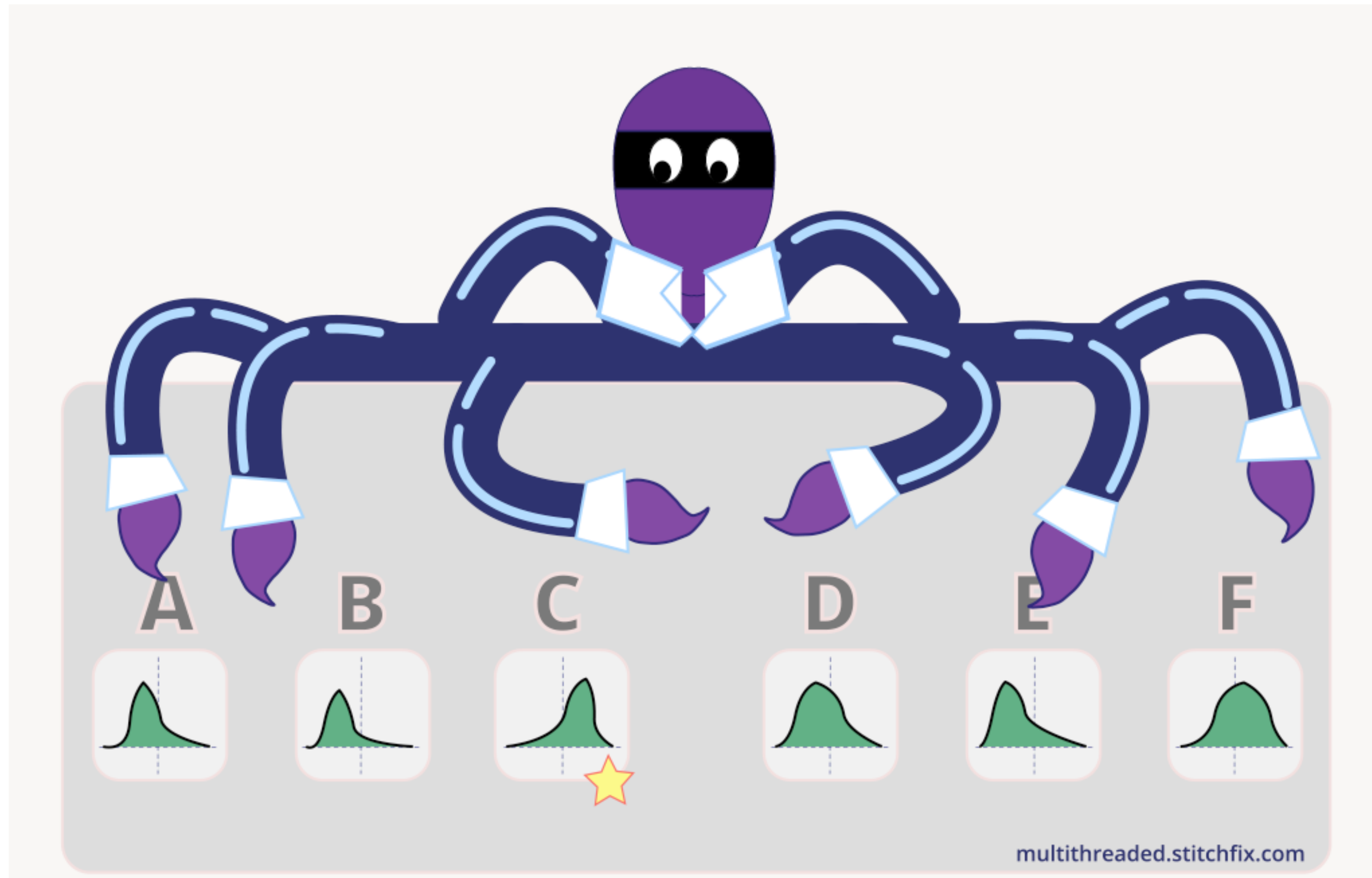


k (number of arms): T (number of steps):

[Source](#)

Next Up: Model-Based Approach

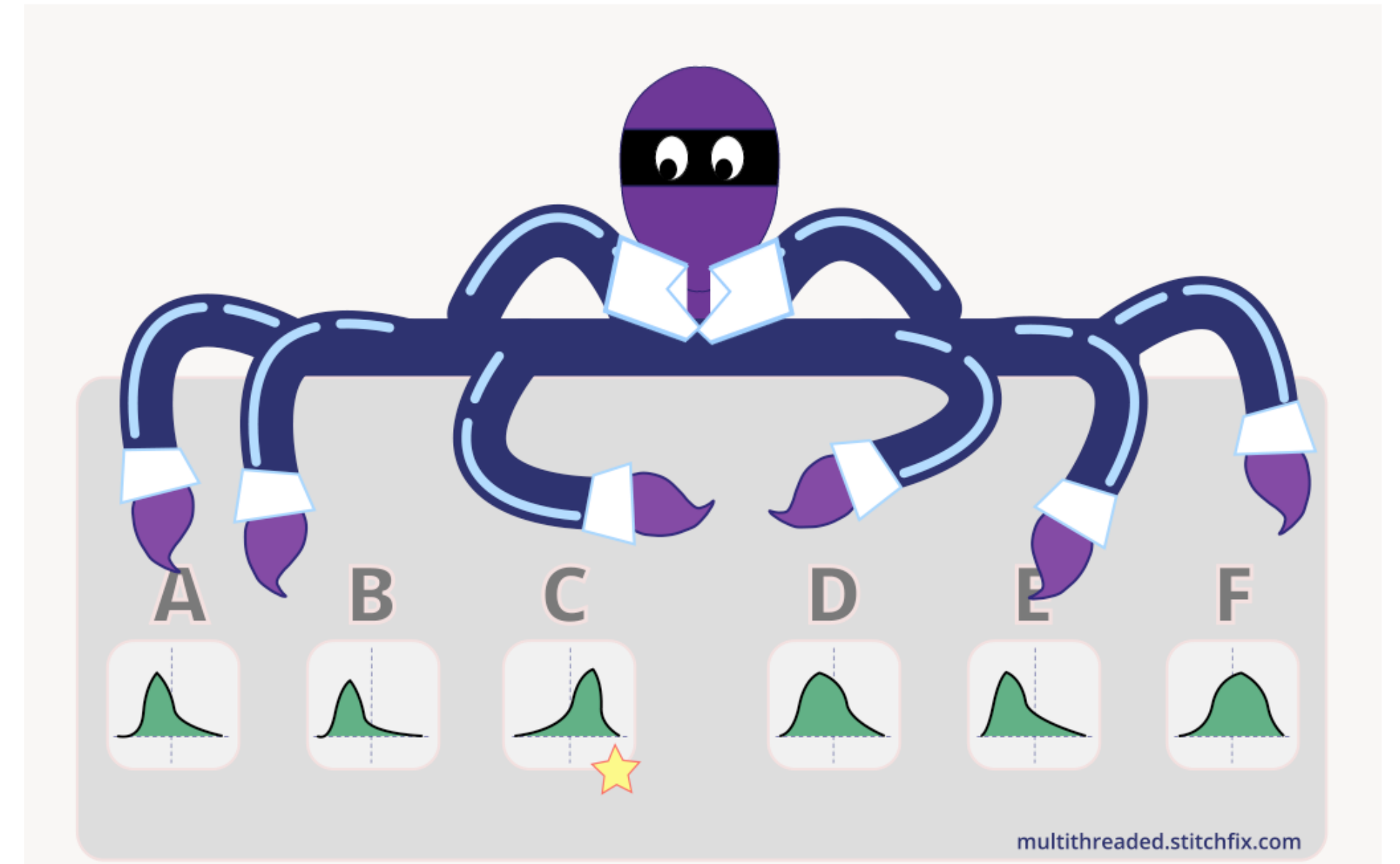
Remember MAB Idea



Source

Model-Based Approach

- $\{p(r | a) \mid a \in \mathcal{A}\}$ is a set of arms, each having its own reward distribution;
- Therefore, let's parametrize this directly by learning the environment's model instead of just mean: $p(r | a) \approx p(r | \theta_a)$
- θ_a are the unknown parameters of the arm a distribution
- For instance, in case of a Bernoulli likelihood, θ_a is the success probability of each arm



Source

Model-Based Approach

- Learn the environment's model instead of just mean: $p(r | a) \approx p(r | \theta_a)$
- We can use Bayesian approach, i.e., treating the data as random, *observed* data as fixed and parameters as random variables
- It allows us to inject rich prior knowledge $\theta_a \sim p_0$ (or stick to fully data-driven approach with improper or weakly informative prior)
- We can then use posterior belief to guide exploration after round t :
$$p(\theta_a) \leftarrow p(\theta_a | r_t) \propto p(r_t | \theta_a) p_0(\theta_a)$$
- $\mathbb{E}_{r \sim p(\cdot | \theta_a)} r$ is a random variable, because θ_a are now random variables
- Moreover, we **gain estimates of success probabilities** in this model, compared to modelling average reward only in UCB

Model-Based Approach

- Example: Bernoulli Bandits

Probability Matching

- After observing data D we can choose an action in the following way:

$$a = \operatorname{argmax}_{a \in \mathcal{A}} \mathbb{E}_{\theta_a \sim p(\theta_a)} \mathbb{E}_{r \sim p(\cdot | \theta_a)} r = \operatorname{argmax}_{a \in \mathcal{A}} \mathbb{E}[r | D, a]$$

- It is exactly the mean reward of the posterior predictive
- **Problem:** We don't use the uncertainty, i.e., no exploration

Probability Matching

- Denote $\mu_a(\theta_a) := \mathbb{E}_{r \sim p(\cdot | \theta_a)} r$
- Let's choose an action with the probability of being optimal

$$\pi(a) = \mathbb{P}[\mathbb{E}_{r \sim p(\cdot | \theta_a)} r > \mathbb{E}_{r \sim p(\cdot | \theta_{a'}, a' \neq a)} r] = \mathbb{P}[\mu_a(\theta_a) > \mu_{a'}(\theta_{a'}) \mid a' \neq a]$$

- This can be rewritten as

$$\pi(a) = \mathbb{P}[\mu_a(\theta_a) = \max_{a' \in \mathcal{A}} \mu_{a'}(\theta_{a'}) \mid D]$$

- I.e., our (stochastic) policy is just the **posterior probability that arm a has the largest average reward across all arms**
- **Problem:** $\pi(a)$ is usually intractable

Thompson Sampling

- Thompson sampling implements probability matching:

$$\pi(a) = \mathbb{P}[\mu_a(\theta_a) = \max_{a' \in \mathcal{A}} \mu_{a'}(\theta_{a'}) \mid D]$$

1. Use Bayes's formula to compute the posterior distribution $p(\theta_a \mid D)$
2. Sample parameters θ_a from this posterior distribution for each arm
3. Select action maximising value on sample: $\hat{a} = \operatorname{argmax}_{a \in \mathcal{A}} \mathbb{E}_{r \sim p(\cdot \mid \theta_a)} r$

Thompson Sampling

Then in this scheme the probability of choosing arm a is

$$\mathbb{P}(\hat{a} = a \mid D) = \mathbb{P}(\operatorname{argmax}_{a \in \mathcal{A}} \mathbb{E}_{r \sim p(\cdot | \theta_a)} r = a \mid D) = \mathbb{P}(\max_{a \in \mathcal{A}} \mathbb{E}_{r \sim p(\cdot | \theta_a)} r = \mu_a(\theta_a) \mid D) = \pi(a)$$

- Then **Thompson Sampling** is exactly an approximating scheme for our desired policy
- It allows to include naturally the exploration and inject some prior knowledge on your MAB task

Under mild conditions on chosen prior (prior should be absolutely continuous w.r.t. true distribution)

Thompson sampling achieves a logarithmic bound!

Next Up: Add Features

Contextual Bandits

1. Now we assume that "not all starts are equal", meaning that instead of a fixed $s \sim \delta_{s_0}$ we have each time a state $s_t \sim p_{state}$.

NB: as usual, states can be both discrete and continuous - in the latter case we rather say that "current state is very close to some observed one".

2. The algorithm observes a "context" $x_t \sim p(\cdot | s_t)$.
3. The algorithm picks an arm a_t from the K possible actions.
4. The reward $r_t \sim p(\cdot | s_t, a_t)$ is realized.

Example: a user with a known "user profile" arrives in each round.

Contextual Bandits

1. Now we assume that "not all starts are equal", meaning that instead of a fixed $s \sim \delta_{s_0}$ we have each time a state $s_t \sim p_{state}$.

NB: as usual, states can be both discrete and continuous - in the latter case we rather say that "current state is very close to some observed one".

2. The algorithm observes a "context" $x_t \sim p(\cdot | s_t)$.
3. The algorithm picks an arm a_t from the K possible actions.
4. The reward $r_t \sim p(\cdot | s_t, a_t)$ is realized.

Example: a user with a known "user profile" arrives in each round.

Very Powerful in Business Applications!

Context

Designing the context is the usual feature engineering. It can include:

1. User's features.
2. Day of the week, time of the day, season or proximity to a major event.
3. The set of feasible actions.
4. Action itself.
5. Lagged optimal actions.

Disjoint LinUCB

- Let $x_{t,a} \in \mathbb{R}^d$ the context summarise information of both the global context x_t (e.g. for user u_t) and arm a .
- The main assumption: $r_{t,a} \mid x_t, a = x_{t,a}^T \theta_a + \epsilon_t$
- Apply ridge regression to derive $\hat{\theta}_{t,a}$ on each step.

Disjoint LinUCB

- $D_a \in \mathbb{R}^{m \times d}$ is a matrix containing contexts which were observed previously for action a .
- $A_a = D_a^T D_a + \lambda I_d$
- $b_a = D_a^T r_{t,a}$
- Ridge Estimate is $\theta_a = A_a^{-1} b_a$
- Expected payoff on each step: $x_{t,a}^T \hat{\theta}_a$ with variance $x_{t,a}^T A_a^{-1} x_{t,a}$.

Algorithm 1 LinUCB with disjoint linear models.

```
0: Inputs:  $\alpha \in \mathbb{R}_+$ 
1: for  $t = 1, 2, 3, \dots, T$  do
2:   Observe features of all arms  $a \in \mathcal{A}_t$ :  $\mathbf{x}_{t,a} \in \mathbb{R}^d$ 
3:   for all  $a \in \mathcal{A}_t$  do
4:     if  $a$  is new then
5:        $\mathbf{A}_a \leftarrow \mathbf{I}_d$  ( $d$ -dimensional identity matrix)
6:        $\mathbf{b}_a \leftarrow \mathbf{0}_{d \times 1}$  ( $d$ -dimensional zero vector)
7:     end if
8:      $\hat{\boldsymbol{\theta}}_a \leftarrow \mathbf{A}_a^{-1} \mathbf{b}_a$ 
9:      $p_{t,a} \leftarrow \hat{\boldsymbol{\theta}}_a^\top \mathbf{x}_{t,a} + \alpha \sqrt{\mathbf{x}_{t,a}^\top \mathbf{A}_a^{-1} \mathbf{x}_{t,a}}$ 
10:   end for
11:   Choose arm  $a_t = \arg \max_{a \in \mathcal{A}_t} p_{t,a}$  with ties broken arbitrarily, and observe a real-valued payoff  $r_t$ 
12:    $\mathbf{A}_{a_t} \leftarrow \mathbf{A}_{a_t} + \mathbf{x}_{t,a_t} \mathbf{x}_{t,a_t}^\top$ 
13:    $\mathbf{b}_{a_t} \leftarrow \mathbf{b}_{a_t} + r_t \mathbf{x}_{t,a_t}$ 
14: end for
```

Disjoint LinUCB

- $D_a \in \mathbb{R}^{m \times d}$ is a matrix containing contexts which were observed previously for action a .
- $A_a = D_a^T D_a + \lambda I_d$
- $b_a = D_a^T r_{t,a}$
- Ridge Estimate is $\theta_a = A_a^{-1} b_a$
- Expected payoff on each step: $x_{t,a}^T \hat{\theta}_a$ with variance $x_{t,a}^T A_a^{-1} x_{t,a}$.

Algorithm 1 LinUCB with disjoint linear models.

```

0: Inputs:  $\alpha \in \mathbb{R}_+$ 
1: for  $t = 1, 2, 3, \dots, T$  do
2:   Observe features of all arms  $a \in \mathcal{A}_t$ :  $\mathbf{x}_{t,a} \in \mathbb{R}^d$ 
3:   for all  $a \in \mathcal{A}_t$  do
4:     if  $a$  is new then
5:        $\mathbf{A}_a \leftarrow \mathbf{I}_d$  ( $d$ -dimensional identity matrix)
6:        $\mathbf{b}_a \leftarrow \mathbf{0}_{d \times 1}$  ( $d$ -dimensional zero vector)
7:     end if
8:      $\hat{\boldsymbol{\theta}}_a \leftarrow \mathbf{A}_a^{-1} \mathbf{b}_a$ 
9:      $p_{t,a} \leftarrow \hat{\boldsymbol{\theta}}_a^\top \mathbf{x}_{t,a} + \alpha \sqrt{\mathbf{x}_{t,a}^\top \mathbf{A}_a^{-1} \mathbf{x}_{t,a}}$ 
10:   end for
11:   Choose arm  $a_t = \arg \max_{a \in \mathcal{A}_t} p_{t,a}$  with ties broken arbitrarily, and observe a real-valued payoff  $r_t$ 
12:    $\mathbf{A}_{a_t} \leftarrow \mathbf{A}_{a_t} + \mathbf{x}_{t,a_t} \mathbf{x}_{t,a_t}^\top$ 
13:    $\mathbf{b}_{a_t} \leftarrow \mathbf{b}_{a_t} + r_t \mathbf{x}_{t,a_t}$ 
14: end for

```

Regret bound: $\tilde{O}(\sqrt{KdT})$,
 where $\tilde{O}(\cdot)$ is the same as $O(\cdot)$ but suppresses logarithmic factors.

Bayesian Interpretation

- Gaussian prior: $p(\theta_a) = \mathcal{N}(0, \tau^2 I_d)$
- On step t for action a :
 - m noisy measurements: $\mathbf{r}_a \sim \mathcal{N}(D_a \theta_a, \sigma^2 I_m)$
 - Posterior distribution: $\theta_a | \mathbf{r}_a \sim \mathcal{N}(\hat{\theta}_a, A_a^{-1})$
 - $A_a = D_a^T D_a + \frac{\sigma^2}{\tau^2} I_d, \quad \hat{\theta}_a = A_a^{-1} D_a^T \mathbf{r}_a$
 - $x_{t,a}^T \theta_a \sim \mathcal{N}(x_{t,a}^T \hat{\theta}_a, x_{t,a}^T A_a^{-1} x_{t,a})$
 - UCB: $x_{t,a}^T \hat{\theta}_a + \alpha \sqrt{x_{t,a}^T A_a^{-1} x_{t,a}}$

Neural UCB

Algorithm 1 NeuralUCB

- 1: **Input:** Number of rounds T , regularization parameter λ , exploration parameter ν , confidence parameter δ , norm parameter S , step size η , number of gradient descent steps J , network width m , network depth L .
- 2: **Initialization:** Randomly initialize θ_0 as described in the text
- 3: Initialize $\mathbf{Z}_0 = \lambda \mathbf{I}$
- 4: **for** $t = 1, \dots, T$ **do**
- 5: Observe $\{\mathbf{x}_{t,a}\}_{a=1}^K$
- 6: **for** $a = 1, \dots, K$ **do**
- 7: Compute $U_{t,a} = f(\mathbf{x}_{t,a}; \theta_{t-1}) + \gamma_{t-1} \sqrt{\mathbf{g}(\mathbf{x}_{t,a}; \theta_{t-1})^\top \mathbf{Z}_{t-1}^{-1} \mathbf{g}(\mathbf{x}_{t,a}; \theta_{t-1}) / m}$
- 8: Let $a_t = \operatorname{argmax}_{a \in [K]} U_{t,a}$
- 9: **end for**
- 10: Play a_t and observe reward r_{t,a_t}
- 11: Compute $\mathbf{Z}_t = \mathbf{Z}_{t-1} + \mathbf{g}(\mathbf{x}_{t,a_t}; \theta_{t-1}) \mathbf{g}(\mathbf{x}_{t,a_t}; \theta_{t-1})^\top / m$
- 12: Let $\theta_t = \text{TrainNN}(\lambda, \eta, J, m, \{\mathbf{x}_{i,a_i}\}_{i=1}^t, \{r_{i,a_i}\}_{i=1}^t, \theta_0)$
- 13: Compute

$$\gamma_t = \sqrt{1 + C_1 m^{-1/6} \sqrt{\log m} L^4 t^{7/6} \lambda^{-7/6}} \cdot \left(\nu \sqrt{\log \frac{\det \mathbf{Z}_t}{\det \lambda \mathbf{I}}} + C_2 m^{-1/6} \sqrt{\log m} L^4 t^{5/3} \lambda^{-1/6} - 2 \log \delta + \sqrt{\lambda} S \right) \\ + (\lambda + C_3 t L) \left[(1 - \eta m \lambda)^{J/2} \sqrt{t/\lambda} + m^{-1/6} \sqrt{\log m} L^{7/2} t^{5/3} \lambda^{-5/3} (1 + \sqrt{t/\lambda}) \right].$$

14: **end for**

Regret bound: $\tilde{O}(\tilde{d}\sqrt{T})$

Next Up: Continuous Actions

A (Too Brief) Intro to Gaussian Processes

- *Riesz Representation Theorem*: A very complex function (continuous linear functional) on Hilbert space can be represented as an affine function of kernelized inputs
- *Kernel Trick*: For a feature map $\phi(x)$ one can define a kernel $\mathbf{K} = \phi(x)^T \phi(x)$, which are positive definite and subset of *Mercer* kernels
- We can introduce a Gaussian Process, which is an infinite set of random variables on a set X , where for each $x \in X$ we have a random variable f_x
- We can view GP as a distribution on a **function space**, such that on some subset $A \subseteq X$, $A = \{x_1, \dots, x_m\}$ it holds that $f_A = [f_{x_1}, \dots, f_{x_m}] \sim \mathcal{N}(\mu_A, \mathbf{K}_{AA})$
- This allows us to model directly **continuous reward functions**

GP-UCB

- Imagine that we work on a continuous action space $\mathcal{A}$, starting from a fixed $s \sim \delta_{s_0}$
- Our task is to maximize the reward $r^* : \mathcal{A} \rightarrow \mathbb{R}$, observing at round t a sample $r_t = r^*(a_t) + \epsilon_t$, $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$ for a chosen action a_t
- Let's model r^* with GP under prior $r \sim GP(m_0, k_0)$, e.g., $GP(0, RBF)$
- Then after observing each round we update our posterior (only means and covariances)

$$\mu_t(a) = k_t(a)^T (\mathbf{K}_t + \sigma^2 \mathbf{I})^{-1} r_{1:t}$$

$$\sigma^2(a) = k_0(a, a) - k_t(a)^T (\mathbf{K}_t + \sigma^2 \mathbf{I})^{-1} k_t(a)$$

$$\text{for } \mathbf{K}_t = [k_0(a_i, a_j)]_{i,j=1}^t \quad k_t(a) = [k_0(a_1, a), \dots, k_0(a_t, a)]$$

GP-UCB

- Then the choice of GP-UCB is simple

$$a_{t+1} = \operatorname{argmax}_{a \in \mathcal{A}} \mu_t(a) + B_t \sigma_t(a)$$

- One can take B_t to balance exploration and exploitation, then for well-calibrated B_t and γ_t maximum information gain after T rounds we have

$$R_T = \mathcal{O}(B_T \sqrt{T \gamma_t})$$

GP-UCB

- Then the choice of GP-UCB is simple

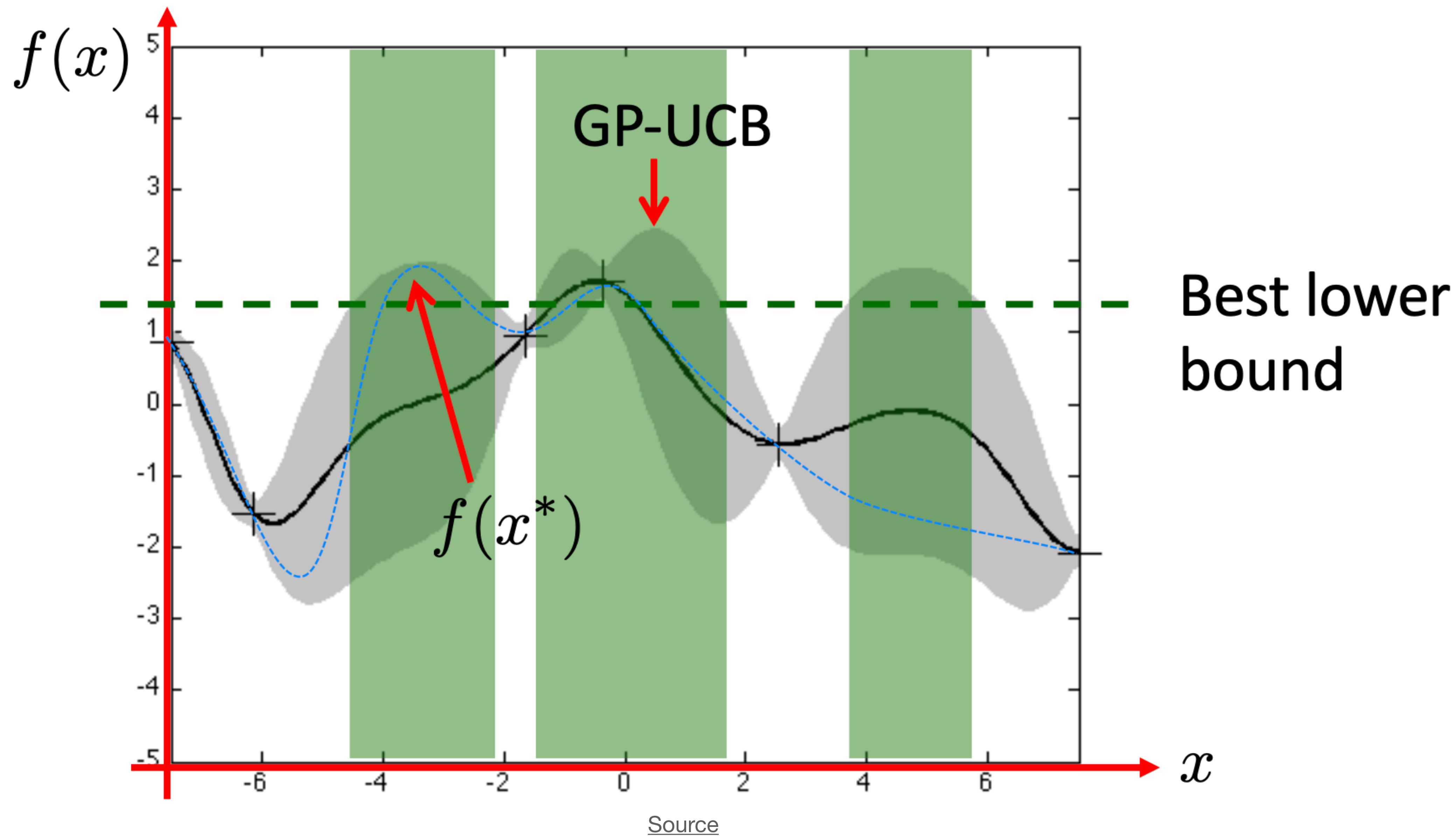
$$a_{t+1} = \operatorname{argmax}_{a \in \mathcal{A}} \mu_t(a) + B_t \sigma_t(a)$$

- In practice, to optimize the decision above we would use usual optimization techniques that work with function evaluations (e.g., `scipy.optimize`)
- One can take B_t to balance exploration and exploitation, then for well-calibrated B_t and γ_t maximum information gain after T rounds we have

$$R_T = \mathcal{O}(B_T \sqrt{T \gamma_t})$$

Often (in complex domains) can beat NNs due to automatically arising well-calibrated uncertainty

GP-UCB



Practical Considerations

- The issue with GPs is the fact that due to inversion of the kernel $\mathbf{K}_{AA}^{-1}$ it scales with $\mathcal{O}(N^3)$ for N observed samples, which can be very slow
- In practice, for a very reasonable choice of RBF Kernel, one could apply Random Fourier Features approximation, which is relatively precise in many domains
- Then you can use a pipeline of RFF + Ridge for computation of the GP approximation, speeding up the computations very significantly

Bandits: Why Need To Model With Reward?

- We still have **reward function**:
 - Imagine that we have **partial observability** $x_t \sim p(\cdot | s_t)$
 - Policy Learning method collects optimal actions on training data $a^*(s_t) = \operatorname{argmax}_{a \in \mathcal{A}} r(s_t, a)$ and then predicts an optimal action $\hat{a}_t = f(x_t)$
 - It can be shown to have **higher expected empirical loss** than the Contextual Bandits (i.e., cost-sensitive optimization), because of loss mismatch problem

Contextual GP-UCB

- Imagine that we work on a continuous action space $\mathcal{A}$ and continuous context space $\mathcal{X}$, which describes $s \sim p_{state}$
- Then we can easily modify GP-UCB by applying a kernel on a product space $\mathcal{X} \times \mathcal{A}$
- For instance, one can take a product kernel $k_0 \times k'_0$ as a prior
- Then we collect the data as $z = (x, a) \in \mathcal{X} \times \mathcal{A}$ and apply the same GP-UCB logic with new input: instead of a we use z
- To sample an action a_{t+1} we would fix the context x_{t+1} that we observe and optimize over a only

Bandits in Practice

- Recommender systems: [Spotify](#), [Netflix](#)
- [Adaptive clinical trials](#)
- [Advertisement](#)
- [COVID-19 border testing](#)
- [Optuna Hyperparameter Tuning](#)
- [Financial Portfolio Optimization](#)
- And many more...

Background

1. Practical RL course by YSDA, week 5
2. Sutton & Barto, Chapter 2
3. DeepMind course, Lecture 2
4. David Silver Course, Lecture 9
5. Introduction to Multi-Armed Bandits

Thank you for your attention!